

Week 1 — Challenge Lab: ASCII Art Printer

☐ Challenge Overview

Name: ASCII Art Printer **Difficulty:** ☆☆☆ Intermediate (for Week 1 level) **Estimated Time:** 20–30 minutes **Skills Practiced:** `print()`, string formatting, creative problem-solving

Requirements

Using **only** `print()` statements, create an ASCII art picture of your choice.

Your art must meet these requirements:

Requirement	Details
Minimum height	At least 8 lines tall
Tools allowed	Only <code>print()</code> — no loops, no variables (yet!)
Subject	Anything you like — house, rocket, animal, robot, tree, etc.
Comments	At least 3 comments labelling the major sections of your art
Title	Print a title above your art (e.g., <code>=== My Rocket ===</code>)

☐ Hints

Hint 1 — Characters Available for ASCII Art

You can use any keyboard characters:

```
/ \ | - _ = + * # @ ! . : ; ~ ^ < > ( ) [ ] { }
```

Hint 2 — Alignment Trick

Use spaces inside your strings to align elements. Count carefully.

```
print("    /\\"")    # forward slash + backslash for a peak
print("   /  \\"")    # wider
print("  /    \\"")    # wider still
print(" /      \\"")    # base
```

Produces:

```
  /\
 /  \
/    \
```

/ ____ \

Hint 3 — Common Patterns

```
# Rocket tip
print("    /\\"")
print("   /  \\"")

# Box / building walls
print(" |    |")
print(" |    |")

# Ground or base
print(" -----")

# Simple tree
print("    *")
print("   ***")
print("  *****")
print("    |")
```

Hint 4 — Escaping Backslashes

In Python, `\` inside a string is the **escape character**. To print a single backslash, you need to type two: `\\`.

```
print("\\")      # Prints: \
print("/\\")     # Prints: /\
```

Hint 5 — Plan Before You Type

Sketch your art on paper first, then transcribe it into `print()` statements. Planning = faster execution.



Planning Strategy

1. **Choose your subject** — pick something simple: a house, tree, rocket, face, or letter
2. **Sketch it on paper** — use graph paper or a notebook, one character per cell
3. **Identify sections** — break the art into top, middle, bottom sections (label with comments)
4. **Code from top to bottom** — one `print()` per row of art
5. **Run and adjust** — run after every few lines to check alignment

⚠ Common Pitfalls

Pitfall	Solution
Backslash <code>\</code> doesn't print	Use <code>\\</code> inside the string
Art is misaligned	Count spaces carefully; use a monospace font in your editor

Pitfall	Solution
Lines too short or too long	Run frequently and compare each line to your sketch
Forgetting quotes	Every string must be inside " . . . "
Mixing up (pipe) and l (lowercase L)	In most ASCII art, (Shift+backslash) is used for vertical lines

□ Example Art — A Simple House

Note: This example is provided so you can see the technique. Create something **original** for your submission — don't copy this.

```
# === ASCII Art: A House ===
# Top section: roof

print("      /\\"")
print("     /  \\"")
print("    /    \\"")
print("   /      \\"")
print("  /______\\")

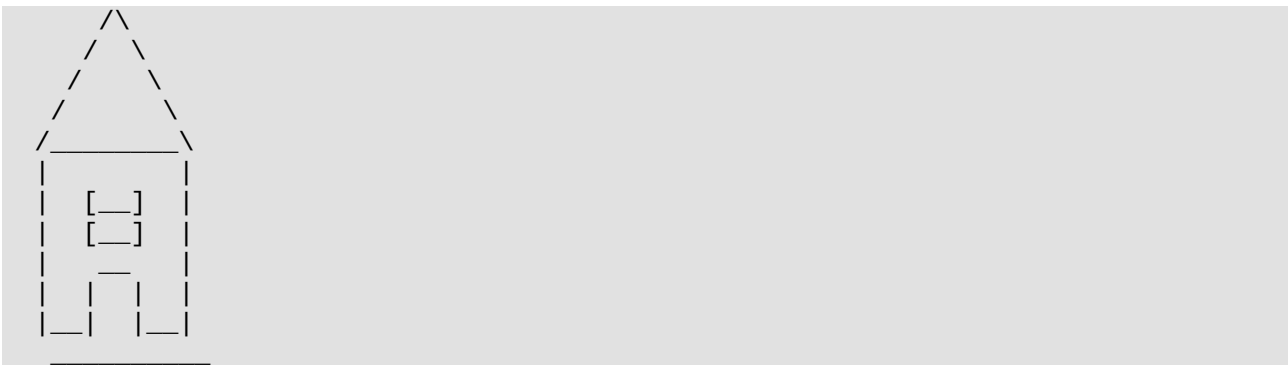
# Middle section: walls and window

print(" |      |")
print(" |  [__]  |")
print(" |  [__]  |")

# Bottom section: door and ground

print(" |  _  |")
print(" | |  | |")
print(" |__|  |__|")
print("  _____")
```

Output:



□ Bonus Challenges

Once your main art is complete, try these extensions:

Bonus 1 — Animated Title

```
# Print a decorative animated-style title
print("'" * 30)
print("'" + " " * 28 + "'")
print("'" + "    MY AWESOME ASCII ART    ".center(28) + "'")
print("'" + " " * 28 + "'")
print("'" * 30)
```

Bonus 2 — Side-by-Side Art

Try placing two small ASCII items next to each other (e.g., two trees, a sun and a cloud).

Bonus 3 — Your Initials in Block Letters

Create your initials as large block letters:



Bonus 4 — A Scene

Combine multiple elements: ground + house + tree + sun = a complete scene.

☐ Success Criteria

Criterion	Check
At least 8 print() lines for the art itself	
Has a title printed above the art	
Has at least 3 comments labelling sections	
No loops or variables used (Week 1 constraint)	
Art is recognizable and intentional	
Script runs without errors	

☐ Cross References

Previous Topic	Solutions
Next Topic	Revision Sheet
Related Concepts	String repetition (Week 2), Loops for pattern generation (Week 3)
Week README	Week 1 Overview

← [Solutions](#) | Next: [Revision Sheet](#) →